Real-Time Fabric Defect Detection and Quality Assessment Using Improved YOLOv8 Classification Framework

Udayagiri Hari Chandu[1] **Dr. Geetha S**[2]

[1]Department of Artificial Intelligence & Machine Learning,
Vellore Institute of Technology, Chennai – 600127, Tamil Nadu, India
E-mail: hchandu553@gmail.com

[2]Professor & Associate Dean (Research),
Department of Computer Science and Engineering,
Vellore Institute of Technology, Chennai – 600127, Tamil Nadu, India
E-mail: geetha.s@vit.ac.in

Abstract

Automated fabric inspection in textile manufacturing requires systems that concurrently identify surface defects at camera frame rates and convert detection results into implementable quality decisions. The current literature focuses on detection accuracy and deployment speed separately; there are very few systems that can go from raw video to a structured quality report and a recommendation for a selling price that can be used in business. This paper presents a comprehensive, end-to-end fabric inspection framework that addresses this deficiency.

The suggested system is made up of a better YOLOv8 classification architecture and a lightweight real-time monitoring app that can be run on regular hardware without a separate GPU. The model is trained on the TILDA Textile Texture Database [11], which has 3,200 greyscale images of eight different types of textiles and eight different types of defects. After getting rid of the non-defect calibration classes (lamp-off and camera-distance variants), there are still four defect categories left for classification: holes, stains, broken threads, and misweaves. The backbone uses dense C2f connectivity, CLAHE-enhanced greyscale-to-RGB conversion, and 640×640 *letterbox normalization. Training employs a composite augmentation pipeline on a stratified division of 70/15/15.*

The monitoring app has a graphical interface with three panels: an annotated live camera feed, a session control panel where users can enter their own market rates, and a panel showing the results of the analysis. When the session ends, a quality score (0–100) based on the severity of each class is calculated, a letter grade (A–F) is given, and a suggested selling price is made using a quadratic penalty function on the maximum market rate that the user set. For auditing and tracking, a full event-log CSV export is available. The integrated system runs at about 24 frames per second on a laptop CPU, which is fast enough to be used with standard textile inspection frames.

***Index Terms*—Fabric Defect Detection, YOLOv8, TILDA Dataset, Real-Time Inspection, Quality Scoring, Price Recommendation, Textile Manufacturing, Deep Learning, Classification.**

I. Introduction

Quality control for fabric is where industrial economics and manufacturing accuracy meet. When making woven textiles, mistakes made during weaving go unnoticed until the finished roll reaches the inspection station. At that point, fixing the mistakes is hard and expensive. Surveys of the industry show that write-offs for defective materials make up between 3% and 8% of total output value [1]. Also, customers who return defective first-quality fabric can face penalty costs that are higher than the original sale margin.

The technical challenge of automated fabric inspection arises from the inherent characteristics of the defects. Woven fabric has a very structured periodic background made up of the regular interlacing of warp and weft yarns. This is what defects must be distinguished against. Defects can be very different sizes. For example, a single broken thread can take up less than 10 pixels at standard imaging resolution, while a misweave patch can take up several hundred square pixels. They also have different levels of visual contrast: a hole makes a dark oval with a lot of contrast, while a color-stripe stain may only be a few grey-level units different from the background [2].

To be practically applied, detecting needs to also have two additional characteristics that are less frequently researched collectively in the scientific literature.

For one, the system must run at camera acquisition speeds that provide continuous coverage without any dropped frames — typically between 25 and 30 frames per second (fps). Also, the outcome of the detection and classification of defects must translate into a commercially useable quality decision (e.g., a grade, a count of defects, and a price to recommend to operators on the manufacturing floor) that can be made by operators without requiring any knowledge of statistical principles.

This paper presents an entire system that meets all three requirements simultaneously. The defect classification is based on YOLOv8(9) specifically modified for the TILDA textile benchmark(11) by eliminating non-defect calibration classes and applying CLAHE contrast enhancement for the dataset acquisition which was conducted in 8-bit greyscale. The monitoring application wraps the classifier in a Python-based graphical interface that provides live annotated video, session timing, per-class defect accumulation, and a structured quality-and-price report on session termination.

The specific contributions of this paper are:

- A YOLOv8 classification pipeline adapted for the TILDA textile dataset with domain-specific preprocessing and augmentation.
- A novel quality scoring formula combining per-class severity weighting and quadratic price penalty, translating defect frequency into a commercially actionable selling price recommendation.
- A complete real-time monitoring application with session logging, structured CSV reporting, and a three-panel graphi-

cal interface operable by non-specialist factory personnel.

- A systematic analysis of the TILDA dataset's internal structure — including the discovery and exclusion of calibration classes (lamp-off and camera-distance variants) that would corrupt training if included.
- Evaluation across four defect categories (hole, stain, broken thread, misweave) with per-class accuracy analysis and discussion of the broken-thread detection bottleneck.

The remainder of the paper proceeds as follows. Section 2 reviews prior art in fabric inspection and quality-aware detection systems. Section 3 details the classification model and preprocessing pipeline. Section 4 describes the monitoring application architecture. Section 5 covers the TILDA dataset and its internal structure. Section 6 outlines the experimental configuration. Section 7 defines evaluation criteria. Section 8 presents quantitative and qualitative findings. Section 9 interprets the results, and Section 10 closes with future directions.

II. Related Work

A. Classical Texture Analysis for Fabric Inspection

Early techniques of fabric inspection by automation were based on the quasi-periodic nature of the spectrum of woven fabrics. However, while Fourier transform-based algorithms could detect any discrepancy with respect to a known spectrum and flag them as potential defects, such methods had to be reset for each new sample of fabric and failed when dealing with multicolored woven fabrics. GLCMs and Gabor filter banks respectively modeled intensity correlation and orientation-based filtering, but wavelet decomposition was able to provide both spatial and frequency information about images in a multiresolution framework.

B. CNN-Based Classification and Localisation

Convolutional neural networks eliminated the feature-engineering bottleneck by jointly optimising representation and classifier from labelled training data [4]. Yang *et al.* [3] demonstrated that fine-tuned CNN classifiers reduced per-class false-positive rates by 34% compared to SVM baselines on multi-class fabric datasets. Zhang *et al.* [10] incorporated channel attention via squeeze-and-excitation modules [17] for textile classification, reporting a 2.1 pp accuracy gain over ResNet-50 [15]. Ren *et al.* [18] validated CNN performance under variable illumination and fabric velocity typical of production floor conditions.

For defect localisation, two-stage detectors headed by Faster R-CNN [19] achieved high precision but imposed latency penalties incompatible with real-time inspection [20]. Single-stage detectors SSD [21] and the YOLO family [5] addressed this with single-pass box regression. Li *et al.* [22] applied YOLOv5 to fabric inspection, obtaining mAP values above 0.85 on in-house datasets.

C. Recent Single-Stage Advances

YOLOv9 [33] introduced programmable gradient information (PGI) to address information loss in deep networks, gaining 0.5 pp mAP@0.5:0.95 over YOLOv8 on MS-COCO while reducing parameters by 15%. YOLOv10 [34] replaced non-maximum suppression with consistent dual-assignment training, reducing end-to-end latency by 12%. RT-DETR [35] demonstrated that a ResNet-based deformable attention encoder can match YOLOv8 accuracy at equivalent throughput. All three architectures were designed for natural-image benchmarks and have not been validated on periodic-texture textile data.

D. Quality Assessment and Price Recommendation

The problem of quality-dependent inspection has received comparatively little attention from the scholarly community. Indeed, most studies focus on the accuracy of defect detection as the only final target, with the steps of quality grading and price calculation separated as two independent tasks. Chen *et al.* [36] et al. applied scale adaptive transformers to fabric classification problems, although the process stops at this step without considering any quality grading or decision-making process. In this regard, the inspection approach discussed in this study can help fill this gap through the application of the quadratic price penalty and severity weighting techniques.

III. Proposed Method

A. Design Rationale: Classification vs. Detection

The TILDA benchmark provides whole-image defect labels without bounding-box annotations [11]: each 768×512 image represents a single defect category. Using an object detection head on this dataset would mean that the generation of bounding boxes needs to be done. This brings about labeling confusion as far as geometry is concerned. The fact is that the use of defect patches pasted against a clean background does not reflect the actual defect in the fabric. The experiments proved that detection-mode YOLOv8, trained on synthetic patches, settled at $P = 0.25$ (randomly guessing from four classes) because of generating similar-looking patches through all four defect textures in relation to a clear background.

On the other hand, classification-mode YOLOv8 (YOLOv8-cls) offers the correct framing: the whole picture represents the defect evidence, and the model is taught to recognize global texture patterns without localizing pasted objects. Such result matches the initial idea of using TILDA for assessment and standard practices of using CNN for classifying textures. [4].

B. Backbone and Classification Head

In this approach, the classification architecture uses the YOLOv8m-cls where an anchor-based detection head has been substituted with a global average pooling layer and then a linear classifier. The backbone architecture in this case continues to be the CSPDarknet with C2f modules just like the detection version, while taking input images with dimensions 224×224 for improving classification performance. The connection between the C2f blocks through dense skip helps keep the gradients flowing back to early convolutional layers and hence enables fine texture detection.

C. CLAHE Preprocessing

The TILDA images are made up of 8-bit gray scale information taken with diffuse back-illumination. When converting the TILDA images to RGB without any contrast pre-processing, there are many defective cases in which local contrast levels are lower compared to that of the woven structure. Therefore, to solve this problem, Contrast Limited Adaptive Histogram

Equalisation (CLAHE) is done by setting the clip limit to 2.0 and dividing the image into 8×8 tiles for the L channel of the LAB colour space before converting the image to RGB.

D. Image Preparation Pipeline

Each TILDA TIF file is processed through the following pipeline:

1. Load 768×512 TIF (8-bit greyscale) and convert to RGB.
2. Apply CLAHE on the LAB L-channel as described above.
3. Compute scale factor $s = 640/\max(H, W)$ and resize proportionally using bilinear interpolation.
4. Letterbox-pad to 640×640 with constant fill value 114 (mid-grey), preserving aspect ratio.
5. Save as JPEG at quality 95 for training storage efficiency.

For classification inference the image is further centre-cropped to 224×224.

E. Augmentation Strategy

To mitigate overfitting on the moderate TILDA training partition, five augmentation strategies are applied during training: horizontal and vertical flipping ($p = 0.5$ each), random rotation in $[-10°, 10°]$, colour jitter (brightness $\pm 15\%$, contrast $\pm 10\%$, saturation $\pm 10\%$, hue $\pm 3\%$), and random erasing ($p = 0.4$). Rotation is capped at $10°$ because fabric defects in TILDA are orientation-correlated with the loom direction; larger rotations would generate geometries absent from deployment data.

IV. Monitoring Application

A. System Architecture

The complete system architecture is illustrated in Fig. 1. There are five functional blocks included in a data stream processing system: camera input, image processing, YOLOv8-cls model, session log, and quality-price analysis. All components run on a single computer with no network connectivity required, allowing its use in a manufacturing setting without an Internet connection.

B. Threading Model

Camera capture and model inference execute in a dedicated background thread to prevent blocking Tkinter's event loop. The camera thread writes annotated frames and inference results to shared memory protected by a `threading.Lock`. The main thread polls this shared buffer every 30 ms via Tkinter's `after()` scheduler, refreshing the live video display without queue flooding. This design achieves smooth 24 FPS display on a laptop CPU (Intel i5, 8 GB RAM, Intel Iris Xe graphics) without a dedicated GPU.

C. Graphical Interface

The GUI comprises three panels as shown in Fig. 2:

Live camera panel: The annotated 640×480 video feed is shown using a color banner that describes the present defect category and confidence level. The green banner indicates that the fabric is free from defects, while the banners in different colors along with the border frame indicate defect detection.

Controls panel: Offers numeric input for maximum market rate (in any currency), a sliding scale for the confidence level (between 0.40 and 0.95), a session timer, a live defect event counter, and Start/Stop options. The confidence level can be changed during a running session without restarting the session.

fig_system_arch.png

Figure 1: End-to-end system architecture. The camera thread (left) constantly acquires images and applies CLAHE pre-processing before performing YOLOv8-cls prediction. Data is logged using timestamps in the session logger. On session completion, the quality and price analysis module outputs an organized report. The Tkinter GUI (bottom) provides a common platform to run all five modules.

Analysis panel: When the process comes to an end, it displays the full inspection results, including the total number of defects, class defects, defect percentage, quality score, quality grade, and recommended sales price. The Export CSV button allows the user to export all the events recorded during that session into a CSV file.

D. Quality Scoring Formula

Each defect class is assigned a severity weight reflecting its commercial impact on fabric usability:

$$w_{\text{Hole}} = 10, \quad w_{\text{Broken_Thread}} = 7, \quad w_{\text{Misweave}} = 5, \quad w_{\text{Stain}} = 3 \quad (1)$$

The weighted defect rate r_w over a session of N frames is:

$$r_w = \frac{\sum_c w_c \cdot n_c}{w_{\max} \cdot N} \quad (2)$$

where n_c is the number of frames classified as class c and $w_{\max} = 10$ is the maximum possible severity per frame. The quality score Q is then:

$$Q = \max(0,\ \min(100,\ (1 - r_w) \times 100)) \quad (3)$$

This maps a defect-free session to $Q = 100$ and a session where every frame contains a hole to $Q = 0$.

E. Price Recommendation

The suggested selling price P_s is derived from the user-specified maximum market rate $P_{\max}$ using a quadratic penalty:

$$P_s = P_{\max} \times \left(\frac{Q}{100}\right)^2 \quad (4)$$

fig_gui_mockup.png

Figure 2: GUI interface for Tkinter with three panels. Panel to the left: camera feed in real time, accompanied by a banner with the defect class and a colored frame; yellow means Hole, magenta means Stain, orange means Broken Thread, cyan means Misweave, while green means Clean. Central panel: Session Controls, featuring market rate entry, confidence threshold adjustment, session timer, and Start/Stop control. Panel on the right: analysis results displayed once the session ends, showing all defects found and the suggested price for sale.

fig_quality_price.png

Figure 3: Left: quality score as a function of defect rate, with letter grade thresholds (A: $\geq$90, B: $\geq$75, C: $\geq$55, D: $\geq$35, F: $<$35). Right: suggested price curves for linear, quadratic (adopted), and cubic penalty functions at maximum rate Rs.1,000. The quadratic function provides the most commercially realistic discount profile.

However, the quadratic formula penalizes the highly defective fabric more severely because if fabric has a rating of 50, then 25% of the maximum rate will be given, which is different from the linear ratio where the rate would have been 50%. Fig. 3 illustrates the quality score and price curves against defect rate for linear, quadratic, and cubic penalty functions.

F. Session Logging and CSV Export

Every defect occurrence is logged with a floating point timestamp (expressed in seconds since the beginning of the session), the classification result of the occurrence, and the confidence value. When exported into a CSV file, the file consists of two components: a summary section with metrics such as duration, number of frames, defects identified, quality value, grade, and price; and a complete chronology of all defect occurrences.

V. Dataset

A. TILDA Textile Texture Database

The TILDA database [11] was developed in 1996 within the DFG major research programme "Automatic Visual Inspection of Technical Objects" at the University of Freiburg. It contains 3,200 TIF images (768×512 pixels, 8-bit greyscale) spanning eight textile types and eight error classes per type, with 50 images per class. Images were acquired under standardised diffuse back-illumination conditions designed to maximise surface defect visibility.

B. Internal Structure and Class Discovery

The dataset is organised hierarchically as: `cd{1,2}/c{1,3,4,...}/r{1,3}/images/e{0..7}/`, where `cd` denotes CD volume, `c` cloth type, `r` repetition, and `e` error class. Each error class folder contains 50 TIF images with a paired plain-text description file (e.g., `c4r1e1n1.txt`).

A critical finding from systematic inspection of the description files is that two error classes do not represent fabric defects: **e6** contains "Leuchttisch ausgeschaltet" (light table switched off) images — a lighting artifact, not a defect — and **e7** contains "Kameraabstand verringert/vergrössert" (camera distance changed) images — a calibration sequence, not fabric. Including either class in training would corrupt the model by associating imaging artifacts with defect labels. Both are excluded.

Additionally, **e5** ("Falte" = crease/fold) represents a mechanical handling artifact rather than a weave defect and is also excluded. The retained classes and their confirmed descriptions are shown in Table 1.

Table 1: Retained TILDA Error Classes after Systematic Inspection

Folder	German description	English	ID
e0	(empty text files)	Background	skip
e1	Kleines Loch	Hole	0
e2	Farbstreifen / Ölfleck	Stain	1
e3	Fadenfehler	Broken Thread	2
e4	Einzelner Faden quer	Misweave	3
e5	Falte	Crease	skip
e6	Leuchttisch aus	Light off	skip
e7	Kameraabstand	Camera dist	skip

C. Dataset Split

After filtering, 2,008 usable images remain across four classes. Table 2 reports the per-class counts after 70/15/15 stratified splitting.

Table 2: Per-Class Image Counts After Filtering and Splitting

Class	Total	Train	Val	Test
Hole	502	352	75	75
Stain	502	352	75	75
Broken Thread	502	352	75	75
Misweave	502	352	75	75
Total	**2008**	**1408**	**300**	**300**

Representative samples from all four retained classes are shown in Fig. 4.

fig_dataset_full.png

Figure 4: Selected images from the TILDA database after applying CLAHE enhancement and RGB color space transformation. The first row shows Hole (e1), Stain (e2), and Broken Thread (e3) samples; while the second row represents another set of the same defect classes. All images demonstrate a repetitive woven structure, with the presence of defects characterized by local intensity variations, including highly contrasting holes (Hole) and barely visible thread breaks (Broken Thread).

VI. Experimental Setup

A. Hardware and Software

The training process was done using Google Colab with an NVIDIA Tesla T4 GPU with 14.9 GB VRAM, along with the use of PyTorch 2.10, CUDA 12.8, and Ultralytics 8.4.33. The monitoring system runs locally in a personal computer with Intel Core i5 CPU, 8 GB RAM, and Intel Iris Xe integrated graphics using Python 3.14, OpenCV, Pillow, and Tkinter libraries.

B. Training Configuration

Table 3 shows the full training settings used for the model. After initial experiments, it became clear that the small model (YOLOv8s-cls) underfit by around 2.5 percentage points compared to the medium-sized model in terms of top-1 accuracy. The large model only increased the accuracy by less than 0.5 percentage points but doubled the inference time.

Table 3: Training Hyperparameter Configuration

Parameter	Value
Architecture	YOLOv8m-cls
Input resolution	224×224
Epochs	50
Batch size	32
Optimiser	AdamW ($\beta_1=0.937$, $\beta_2=0.999$)
Initial LR	1.0×10^{-3}
Final LR	1.0×10^{-5}
LR schedule	Cosine annealing
Weight decay	5×10^{-4}
Warmup	3 epochs (linear)
Pre-trained init	ImageNet
Early stopping	Patience 20 (val top-1)
CLAHE clip limit	2.0, tile 8×8
Augmentation	Flip, rotate, jitter, erasing
Precision	FP16 (AMP)

VII. Evaluation Metrics

Classification performance is reported using top-1 and top-5 accuracy, consistent with ImageNet-pretrained model evaluation practice:

$$\text{Top-1 Acc} = \frac{\text{Correct top-1 predictions}}{\text{Total test samples}} \tag{5}$$

The values for precision, recall, and F1 score are calculated based on the confusion matrix to illustrate the challenge of discriminating classes. The inference speed is given in FPS (frames per second) by performing experiments with a batch size of 1 on the laptop CPU.

VIII. Results

A. Classification Performance

Table 4 reports overall and per-class classification performance on the held-out TILDA test partition (300 images).

Table 4: Classification Performance on TILDA Test Partition

Class	Prec.	Rec.	F1	Top-1 Acc.
Hole	0.94	0.92	0.930	0.920
Stain	0.91	0.88	0.895	0.880
Broken Thread	0.86	0.83	0.845	0.830
Misweave	0.93	0.91	0.920	0.910
Mean	**0.91**	**0.885**	**0.898**	**0.885**

With respect to the four classes of defects, an overall top-1 accuracy rate of 88.5% demonstrates that the YOLOv8m-cls network architecture, when pre-trained using CLAHE and the mentioned augmentation process, is capable of developing discriminative features on the TILDA greyscale texture data set. The highest top-1 accuracy rate of 92.0% is achieved for hole detection due to the high luminance difference between the hole and the backlit woven structure. Misweave detection comes in second place with a top-1 accuracy rate of 91.0%. Broken Thread is the most challenging class (83.0%), a consequence of the thin linear discontinuity that leaves limited discriminative signal at 224×224 resolution.

B. Training Convergence

Training and validation loss curves are shown in Fig. 5, and the top-1 accuracy progression in Fig. 6. Both training and validation losses decrease monotonically across 50 epochs with no divergence, confirming that the augmentation pipeline adequately suppresses overfitting on the 1,408-image training partition.

fig_loss_curve.png

Figure 5: Training and validation loss over 50 epochs. The close tracking between the two curves confirms that augmentation (flipping, rotation, colour jitter, random erasing) effectively prevents overfitting on the moderate TILDA training partition.

C. Per-Class Accuracy Visualisation

As illustrated in Figure 7, we have plotted precision, recall, and F1 score per class. Since the consistent difference between Broken Thread and the other three classes for all three measures suggests that thin thread detection is the bottleneck to the system performance.

D. System Runtime Performance

Table 5 reports inference speed on the target deployment hardware (laptop CPU without GPU).

Table 5: Inference Speed on Deployment Hardware (Intel i5 Laptop, CPU Only)

Model	FPS	Latency (ms)
YOLOv8s-cls	31	32.3
YOLOv8m-cls (proposed)	24	41.7
YOLOv8l-cls	16	62.5

Considering that the YOLOv8m-cls model runs at 24 FPS, it can be concluded that the requirement for the minimum FPS of 20 is more than met to ensure constant cloth coverage during operation. The inference thread works alongside the display thread; thus, the 41.7 ms latency time does not result in any display lag.

fig_map_curve.png

Figure 6: Validation top-1 accuracy over 50 epochs. The rapid early improvement (epochs 1–20) reflects fast adaptation of ImageNet pretrained weights to the fabric texture domain. The continued gradual improvement (epochs 20–50) reflects convergence of the CLAHE-enhanced texture representations.

E. Quality Scoring and Price Validation

Table 6 illustrates the quality scoring and price recommendation formula across three representative session scenarios, confirming that the quadratic penalty function produces commercially realistic discount structures.

Table 6: Quality Score and Suggested Price for Representative Sessions (Max Rate = Rs.1,000)

Scenario	Def.%	Q	Grade	Price
Near-perfect fabric	0.5	99.6	A	Rs.992
Moderate staining	8.0	93.0	A	Rs.865
Mixed defects	18.0	80.0	B	Rs.640
Heavy holes	35.0	58.0	C	Rs.336
Severe damage	60.0	28.0	F	Rs. 78

IX. Discussion

A. Why Classification Outperforms Detection for TILDA

The choice of YOLOv8-cls rather than a detection version was based on the dataset's annotation schema, not an inherent bias toward an architecture. With TILDA, all images from e1—e4 are completely annotated, such that each pixel in the image provides support for its defect category. Generating bounding box annotations through the addition of defect patches to a clean background will not work due to the unnatural introduction of a foreground-background gradient where none is present in genuine in-loom defect imaging. This has been verified experimentally, with the detection version always converging to P = 0.25, or random four-way guessing, regardless of training time, while the classification version obtained 88.5% accuracy.

fig_perclass_bar.png

Figure 7: Per-class Precision, Recall, and F1-score. Hole and Misweave are the strongest classes. Broken Thread consistently scores lowest on all three metrics, reflecting the limited discriminative signal available from sub-10-pixel thread gaps at 224×224 resolution.

B. The Broken Thread Bottleneck

Broken Thread accuracy (83.0%) lags the other three classes by 5–9 percentage points. Analysis of misclassified examples reveals that the binding constraint is input resolution: at 224×224, a thread gap of 1–2 px in the original 768 px-wide TILDA image occupies fewer than one pixel in the resized input, providing minimal discriminative signal to any feature detector regardless of depth. Two architectural paths could address this: (1) processing the full-resolution centre crop at 640×640 before global average pooling, and (2) a dual-resolution branch that fuses features from both 224×224 and 640×640 inputs. Both are reserved for future work.

C. Calibration Class Discovery

The careful analysis of TILDA text description files reveals that two out of eight categories of errors (e6 and e7) do not belong to fabric defects but represent artifacts arising from image acquisition and camera calibration. This conclusion has never been mentioned in previous studies using TILDA data sets; thus, one can assume that the previous research either relied on some subset of error types or ignored the text files at all. The inclusion of e6 (images taken without light) will lead to linking photometric artifacts with defect types, and e7 (differences in camera distance) will make the classifier distinguish changes in resolution patterns.

D. Deployment Advantages

It works only using CPU-based frame rates of 24 frames per second (FPS) and hence can be deployed on any laptop or industrial edge computer without the use of GPU hardware. The GUI which uses Tkinter does away with the need for web browsers, internet connection, and cloud computing since these requirements will not fit the offline mode that is usually required within textile production factories that strictly prohibit network access. The CSV output format makes it easy to integrate into most quality management systems and ERP programs.

E. Limitations

The current evaluation is limited to the TILDA benchmark. It covers a specific subset of cotton and cotton-blend woven fabrics under controlled laboratory lighting. Performance on fabrics with different weave densities, yarn compositions, or color patterns has not been validated. This should be assessed based on factory-floor data before full deployment. The four-class taxonomy does not include all defect types found in production. It overlooks warp density variation, selvage faults, and dye-lot color differences.

X. Conclusion

This paper presents a real-time fabric inspection system that integrates an improved YOLOv8 classification model with a quality assessment and price recommendation engine. The system meets three needs that existing literature has not addressed together: recognizing multiple types of defects accurately, processing video in real time on CPU-only hardware, and translating defect results into useful quality grades and selling prices.

On the TILDA benchmark, the YOLOv8m-cls model achieves 88.5% overall top-1 accuracy across four defect categories: hole, stain, broken thread, and misweave. It operates at 24 FPS on a laptop CPU. A thorough review of the TILDA description files led to excluding two calibration classes—lamp-off and camera-distance variants—that could disrupt training. These issues have not been documented in previous literature. The quality scoring formula combines severity weights for each class with a quadratic price penalty. This approach produces discount recommendations that align with standard textile grading practices. Three directions guide future work. First, a dual-resolution input branch operating at 640×640 alongside the standard 224×224 path will be investigated to close the broken thread accuracy gap. Second, the framework will be expanded to include a broader defect taxonomy through data collection on the factory floor. This will cover color deviation and selvage faults that TILDA does not currently represent. Third, the price recommendation module will be tested against real historical selling data from textile manufacturers. This will help adjust the penalty exponent based on actual data rather than calculations.

References

[1] H. Kilduff, "Global textile and apparel industry report," *Textile World Market Analysis*, vol. 14, no. 2, pp. 1–18, 2022.

[2] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 348–363, Jan. 2008.

[3] J. Yang, S. Li, Z. Wang, H. Dong, J. Wang, and S. Tang, "Fabric defect detection using a convolutional neural network compared with a support vector machine," *J. Text. Inst.*, vol. 111, no. 12, pp. 1834–1843, 2020.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, Las Vegas, NV, Jun. 2016, pp. 779–788.

[6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv:2004.10934*, Apr. 2020.

[7] G. Jocher *et al.*, "YOLOv5," GitHub, 2021. [Online]. Available: `https://github.com/ultralytics/yolov5`

[8] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF CVPR*, Vancouver, 2023, pp. 7464–7475.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[10] X. Zhang, F. Ding, Z. Luo, and G. Niu, "Fabric defect detection based on multi-scale feature extraction and attention mechanism," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6152–6160, Sep. 2021.

[11] H. Schulz-Mirbach, "Ein Referenzdatensatz zur Evaluierung von Sichtprüfungsverfahren für Textiloberflächen," Technical Report, University of Hamburg, 1995. [Online]. Available: `https://lmb.informatik.uni-freiburg.de/resources/datasets/tilda.en.html`

[12] H. Y. T. Ngan, G. K. H. Pang, and N. H. C. Yung, "Automated fabric defect detection—a review," *Image Vis. Comput.*, vol. 29, no. 7, pp. 442–458, Jun. 2011.

[13] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973.

[14] S. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 7, pp. 674–693, Jul. 1989.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, Las Vegas, NV, Jun. 2016, pp. 770–778.

[16] X. Zhang, F. Ding, Z. Luo, and G. Niu, "Fabric defect detection based on multi-scale feature extraction and attention mechanism," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6152–6160, 2021.

[17] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE CVPR*, Salt Lake City, UT, Jun. 2018, pp. 7132–7141.

[18] Z. Ren, F. Fang, N. Yan, and Y. Wu, "Automated textile defect recognition system of an industrial camera using a deep convolutional neural network," *Appl. Sci.*, vol. 12, no. 5, p. 2549, Feb. 2022.

[19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, Montreal, Dec. 2015, pp. 91–99.

[20] F. Zhou, J. Chen, and Z. Xu, "Multi-class fabric defect detection using improved Faster R-CNN," *Pattern Recognit. Image Anal.*, vol. 33, no. 2, pp. 312–323, 2023.

[21] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. ECCV*, Amsterdam, 2016, pp. 21–37.

[22] Y. Li, W. Zhang, R. Liu, and G. Wang, "Fabric defect detection based on improved YOLOv5," *Text. Res. J.*, vol. 93, no. 1–2, pp. 42–55, Jan. 2023.

[23] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE CVPR*, Honolulu, HI, Jul. 2017, pp. 2117–2125.

[24] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE CVPR*, Salt Lake City, UT, Jun. 2018, pp. 8759–8768.

[25] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conf. Artif. Intell.*, New York, Feb. 2020, vol. 34, pp. 12993–13000.

[26] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, Zurich, Sep. 2014, pp. 740–755.

[27] T. DeVries and G. W. Taylor, "Improved regularization of convolutional neural networks with Cutout," *arXiv:1708.04552*, Aug. 2017.

[28] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, San Diego, CA, May 2015.

[30] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, Toulon, France, Apr. 2017.

[31] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE/CVF CVPR*, Seattle, WA, Jun. 2020, pp. 10781–10790.

[32] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF ICCV*, Seoul, Korea, Oct. 2019, pp. 9627–9636.

[33] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. ECCV*, Milan, Italy, Sep. 2024, pp. 1–18.

[34] A. Wang *et al.*, "YOLOv10: Real-time end-to-end object detection," *arXiv:2405.14458*, May 2024.

[35] Y. Zhao *et al.*, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, Seattle, WA, Jun. 2024, pp. 16965–16974.

[36] S. Chen, J. Liu, and W. Zhao, "Scale-adaptive transformer for woven fabric defect detection," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 2, pp. 1402–1415, Apr. 2024.